Author: Sam O'Connor

Student Number: C22316191

Table of Contents

01 March – Task 1: Initial LED Control 1

03 March – Task 2: LED Indexing 6

05 March – Task 2 Improvement 10

07 March – Task 2 Final Fix 14

09 March – Task 3: Accelerometer Setup 17

11 March – Task 4: Basic Tilt Detection 20

13 March – Task 5: Multi-Axis Control 26

15 March – Task 6: Direction Mapping 33

17 March – Task 7: Pointer Display 47

19 March – Task 8: Filtering 55

21 March – Final System Integration 65

01 March – Task 1: Initial LED Control

Goal:

Light a single LED on the WS2812 ring.

Code:

SPI-based WS2812 implementation using encoded bit patterns.

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html
```

```
[env:nucleo_l432kc]
platform = ststm32
board = nucleo_l432kc
framework = cmsis
build_type = debug
build_flags = -O0
```

```c
#include "stm32l4xx.h"
#include <stdint.h>

#define NUM_LEDS 16

static void clock_init_80mhz(void);
static void gpio_init_spi(void);
static void spi1_init(void);
static void spi1_send8(uint8_t data);
static void delay_cycles(volatile uint32_t t);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_encoded_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_test_pattern(void);

int main(void)
{
    clock_init_80mhz();
    gpio_init_spi();
    spi1_init();

    while (1)
    {
        show_test_pattern();
        delay_us(200000);
    }
}

static void delay_cycles(volatile uint32_t t)
{
    while (t--) __NOP();
}

static void delay_us(uint32_t us)
{
    // Roughly for 80 MHz
    while (us--)
    {
        delay_cycles(20);
```

```c
    }
}

static void clock_init_80mhz(void)
{
    // Enable HSI16
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    // Flash latency for 80 MHz
    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    // PLL config:
    // HSI16 / 1 * 10 / 2 = 80 MHz
    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |   // PLLM = 1
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |   // PLLR = 2
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    // Switch SYSCLK to PLL
    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_spi(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    // PA7 = SPI1_MOSI (AF5)
    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));

    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
    GPIOA->AFR[0] |=  (5U << (7 * 4));

    GPIOA->OTYPER &= ~(1U << 7);
    GPIOA->OSPEEDR &= ~(3U << (7 * 2));
    GPIOA->OSPEEDR |=  (3U << (7 * 2));
    GPIOA->PUPDR &= ~(3U << (7 * 2));
}
```

```c
static void spi1_init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    SPI1->CR1 = 0;
    SPI1->CR2 = 0;

    // SPI clock = 80 MHz / 32 = 2.5 MHz
    SPI1->CR1 =
        SPI_CR1_MSTR |
        SPI_CR1_SSM  |
        SPI_CR1_SSI  |
        SPI_CR1_BR_2;   // /32

    SPI1->CR1 |= SPI_CR1_SPE;
}

static void spi1_send8(uint8_t data)
{
    while (!(SPI1->SR & SPI_SR_TXE)) {}
    *((volatile uint8_t *)&SPI1->DR) = data;
    while (SPI1->SR & SPI_SR_BSY) {}
}

static void ws2812_reset(void)
{
    while (SPI1->SR & SPI_SR_BSY) {}

    // Disable SPI so PA7 can be driven as GPIO low
    SPI1->CR1 &= ~SPI_CR1_SPE;

    // PA7 as normal output
    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (1U << (7 * 2));

    // Drive low
    GPIOA->BSRR = (1U << (7 + 16));
    delay_us(80);

    // Back to SPI AF
    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));
    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
    GPIOA->AFR[0] |=  (5U << (7 * 4));

    SPI1->CR1 |= SPI_CR1_SPE;
}
```

```c
static void ws2812_send_encoded_byte(uint8_t byte)
{
    uint32_t encoded = 0;

    for (int i = 7; i >= 0; i--)
    {
        encoded <<= 3;
        if (byte & (1U << i))
        {
            encoded |= 0b110;
        }
        else
        {
            encoded |= 0b100;
        }
    }

    spi1_send8((encoded >> 16) & 0xFF);
    spi1_send8((encoded >> 8) & 0xFF);
    spi1_send8(encoded & 0xFF);
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    // GRB order
    ws2812_send_encoded_byte(g);
    ws2812_send_encoded_byte(r);
    ws2812_send_encoded_byte(b);
}

static void show_test_pattern(void)
{
    ws2812_reset();

    // LED 0 red, others off
    ws2812_send_pixel(0x00, 0x08, 0x00);

    for (int i = 1; i < NUM_LEDS; i++)
    {
        ws2812_send_pixel(0x00, 0x00, 0x00);
    }

    ws2812_reset();
}
```

Result:
LED illuminated, but output was unstable. Colours were inconsistent and unreliable due to incorrect timing.

03 March – Task 2: LED Indexing

Goal:
Control a specific LED on the ring.

Code:
Modified SPI implementation to address individual LEDs.

```
#include "stm32l4xx.h"
#include <stdint.h>

#define NUM_LEDS 16

static void clock_init_80mhz(void);
static void gpio_init_spi(void);
static void spi1_init(void);
static void spi1_send8(uint8_t data);
static void delay_cycles(volatile uint32_t t);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_encoded_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_single_led(uint8_t led_index);

int main(void)
{
    clock_init_80mhz();
    gpio_init_spi();
    spi1_init();

    show_single_led(4);   // change this to 1,2,3...15 to test positions

    while (1)
    {
    }
}

static void delay_cycles(volatile uint32_t t)
{
    while (t--) __NOP();
}
```

```c
static void delay_us(uint32_t us)
{
    while (us--)
    {
        delay_cycles(20);
    }
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_spi(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    // PA7 = SPI1_MOSI (AF5)
    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));

    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
    GPIOA->AFR[0] |=  (5U << (7 * 4));

    GPIOA->OTYPER &= ~(1U << 7);
    GPIOA->OSPEEDR &= ~(3U << (7 * 2));
    GPIOA->OSPEEDR |=  (3U << (7 * 2));
    GPIOA->PUPDR &= ~(3U << (7 * 2));
```

```c
}

static void spi1_init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    SPI1->CR1 = 0;
    SPI1->CR2 = 0;

    // SPI clock = 80 MHz / 32 = 2.5 MHz
    SPI1->CR1 =
        SPI_CR1_MSTR |
        SPI_CR1_SSM  |
        SPI_CR1_SSI  |
        SPI_CR1_BR_2;

    SPI1->CR1 |= SPI_CR1_SPE;
}

static void spi1_send8(uint8_t data)
{
    while (!(SPI1->SR & SPI_SR_TXE)) {}
    *((volatile uint8_t *)&SPI1->DR) = data;
    while (SPI1->SR & SPI_SR_BSY) {}
}

static void ws2812_reset(void)
{
    while (SPI1->SR & SPI_SR_BSY) {}

    SPI1->CR1 &= ~SPI_CR1_SPE;

    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (1U << (7 * 2));

    GPIOA->BSRR = (1U << (7 + 16));
    delay_us(80);

    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));
    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
    GPIOA->AFR[0] |=  (5U << (7 * 4));

    SPI1->CR1 |= SPI_CR1_SPE;
}

static void ws2812_send_encoded_byte(uint8_t byte)
{
```

```c
    uint32_t encoded = 0;

    for (int i = 7; i >= 0; i--)
    {
        encoded <<= 3;
        if (byte & (1U << i))
        {
            encoded |= 0b110;
        }
        else
        {
            encoded |= 0b100;
        }
    }

    spi1_send8((encoded >> 16) & 0xFF);
    spi1_send8((encoded >> 8) & 0xFF);
    spi1_send8(encoded & 0xFF);
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_encoded_byte(g);
    ws2812_send_encoded_byte(r);
    ws2812_send_encoded_byte(b);
}

static void show_single_led(uint8_t led_index)
{
    ws2812_reset();

    for (uint8_t i = 0; i < NUM_LEDS; i++)
    {
        if (i == led_index)
        {
            ws2812_send_pixel(0x00, 0x08, 0x00);   // on
        }
        else
        {
            ws2812_send_pixel(0x00, 0x00, 0x00);   // off
        }
    }

    ws2812_reset();
}
```

Result:
Able to select LEDs, but LED 4 was skipped and incorrect colours appeared. Issue suspected to be related to timing/protocol.

05 March – Task 2 Improvement

Goal:
Fix LED indexing and colour inconsistencies.

Code:
Introduced LED mapping array and logical indexing.

```
#include "stm32l4xx.h"
#include <stdint.h>

#define NUM_LEDS 16
#define NUM_LOGICAL_LEDS 15

static const uint8_t led_map[NUM_LOGICAL_LEDS] =
{
    0, 1, 2, 3,
    5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
};

static void clock_init_80mhz(void);
static void gpio_init_spi(void);
static void spi1_init(void);
static void spi1_send8(uint8_t data);
static void delay_cycles(volatile uint32_t t);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_encoded_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_single_logical_led(uint8_t logical_index);

int main(void)
{
    clock_init_80mhz();
    gpio_init_spi();
    spi1_init();

    show_single_logical_led(2);   // change this to test 0..14

    while (1)
    {
```

```
    }
}

static void delay_cycles(volatile uint32_t t)
{
    while (t--) __NOP();
}

static void delay_us(uint32_t us)
{
    while (us--)
    {
        delay_cycles(20);
    }
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_spi(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));

    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
```

```
    GPIOA->AFR[0] |=  (5U << (7 * 4));

    GPIOA->OTYPER &= ~(1U << 7);
    GPIOA->OSPEEDR &= ~(3U << (7 * 2));
    GPIOA->OSPEEDR |=  (3U << (7 * 2));
    GPIOA->PUPDR &= ~(3U << (7 * 2));
}

static void spi1_init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    SPI1->CR1 = 0;
    SPI1->CR2 = 0;

    SPI1->CR1 =
        SPI_CR1_MSTR |
        SPI_CR1_SSM  |
        SPI_CR1_SSI  |
        SPI_CR1_BR_2;   // /32

    SPI1->CR1 |= SPI_CR1_SPE;
}

static void spi1_send8(uint8_t data)
{
    while (!(SPI1->SR & SPI_SR_TXE)) {}
    *((volatile uint8_t *)&SPI1->DR) = data;
    while (SPI1->SR & SPI_SR_BSY) {}
}

static void ws2812_reset(void)
{
    while (SPI1->SR & SPI_SR_BSY) {}

    SPI1->CR1 &= ~SPI_CR1_SPE;

    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (1U << (7 * 2));

    GPIOA->BSRR = (1U << (7 + 16));
    delay_us(80);

    GPIOA->MODER &= ~(3U << (7 * 2));
    GPIOA->MODER |=  (2U << (7 * 2));
    GPIOA->AFR[0] &= ~(0xFU << (7 * 4));
    GPIOA->AFR[0] |=  (5U << (7 * 4));
```

```
    SPI1->CR1 |= SPI_CR1_SPE;
}

static void ws2812_send_encoded_byte(uint8_t byte)
{
    uint32_t encoded = 0;

    for (int i = 7; i >= 0; i--)
    {
        encoded <<= 3;
        if (byte & (1U << i))
        {
            encoded |= 0b110;
        }
        else
        {
            encoded |= 0b100;
        }
    }

    spi1_send8((encoded >> 16) & 0xFF);
    spi1_send8((encoded >> 8) & 0xFF);
    spi1_send8(encoded & 0xFF);
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_encoded_byte(g);
    ws2812_send_encoded_byte(r);
    ws2812_send_encoded_byte(b);
}

static void show_single_logical_led(uint8_t logical_index)
{
    uint8_t physical_index = 0xFF;

    if (logical_index < NUM_LOGICAL_LEDS)
    {
        physical_index = led_map[logical_index];
    }

    ws2812_reset();

    for (uint8_t i = 0; i < NUM_LEDS; i++)
    {
        if (i == physical_index)
        {
            // brighter white-ish on-value so bad colour channel matters less
```

```
            ws2812_send_pixel(0x08, 0x08, 0x08);
        }
        else
        {
            ws2812_send_pixel(0x00, 0x00, 0x00);
        }
    }

    ws2812_reset();
}
```

Result:
Indexing improved slightly, but colour issues persisted. Confirmed that the root issue was not indexing but signal timing.

07 March – Task 2 Final Fix

Goal:
Achieve stable WS2812 communication.

Code:
Removed SPI completely and implemented direct GPIO control using DWT cycle counter.

Using WS2 protocol instead

```
#include "stm32l4xx.h"
#include <stdint.h>

#define NUM_LEDS 16
#define WS_PIN    7   // PA7

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_test_pattern(void);

int main(void)
{
```

```c
    clock_init_80mhz();
    gpio_init_ws2812();
    dwt_init();

    while (1)
    {
        show_test_pattern();
        delay_us(200000);
    }
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |   // PLLM = 1
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |   // PLLR = 2
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    // PA7 as general purpose output
    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));   // very high speed
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));
```

```
    // start low
    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    // Hold line low for > 50 us
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    // At 80 MHz:
    // 1 cycle = 12.5 ns
    // T0H ~ 0.4 us  -> 32 cycles
    // T0L ~ 0.85 us -> 68 cycles
    // T1H ~ 0.8 us  -> 64 cycles
    // T1L ~ 0.45 us -> 36 cycles
    // Total bit time = 1.25 us -> 100 cycles

    if (bit)
    {
        // Send '1'
        GPIOA->BSRR = (1U << WS_PIN);          // high
        while ((DWT->CYCCNT - start) < 64) {}  // ~0.8 us
        GPIOA->BSRR = (1U << (WS_PIN + 16));   // low
        while ((DWT->CYCCNT - start) < 100) {} // total 1.25 us
    }
    else
    {
```

```
        // Send '0'
        GPIOA->BSRR = (1U << WS_PIN);          // high
        while ((DWT->CYCCNT - start) < 32) {}  // ~0.4 us
        GPIOA->BSRR = (1U << (WS_PIN + 16));   // low
        while ((DWT->CYCCNT - start) < 100) {} // total 1.25 us
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    // WS2812B uses GRB order
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void show_test_pattern(void)
{
    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        if (i == 0) // CHNAGE THIS LINE TO SWITCH LED (0-15)
            ws2812_send_pixel(0x00, 0x08, 0x00); // LED 1 red
        else
            ws2812_send_pixel(0x00, 0x00, 0x00); // others off
    }

    ws2812_reset();
}
```

Result:
Stable LED control achieved. Colours correct and no skipped LEDs. Major breakthrough in project.

09 March – Task 3: Accelerometer Setup

Goal:
Read acceleration data from BMI160 sensor.

Code:
I2C communication implemented with register reads and UART debug output.

```
#include "eeng1030_lib.h"
#include <stdio.h>
#include  <errno.h>
#include  <sys/unistd.h> // STDOUT_FILENO, STDERR_FILENO
#include "i2c.h"
void setup(void);
void initI2C(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

// The following timeout value was determined by
// trial and error running the chip at 72MHz
// There is a little bit of 'headroom' included
#define I2C_TIMEOUT 20000
uint16_t response;
int16_t x_accel;
int32_t X_g;
int main()
{
    setup();
    initSerial(9600);
    ResetI2C();
// Take accelerometer out of power down mode
    I2CStart(0x69,WRITE,2);
    I2CWrite(0x7e);
    I2CWrite(0x11);
    I2CStop();
    delay(1000000); // wait for power up.
    while(1)
    {
        GPIOB->ODR |= (1 << 3); // set port bit for logic analyser debug
        I2CStart(0x69,WRITE,1); // Write the address of the
        I2CWrite(0x12);         // register we want to talk to
        I2CReStart(0x69,READ,2);// Switch to read mode and request 2 bytes
        response=I2CRead();     // read low byte
        x_accel=response;
        response=I2CRead();     // read high byte
        x_accel=x_accel+(response << 8); // combine bytes
        I2CStop();  // end I2C transaction
        GPIOB->ODR &= ~(1 << 3); // clear port bit for logic analyser debug
        X_g = x_accel;  // promote to 32 bits and preserve sign
        X_g=(X_g*981)/16384; // assuming +1g ->16384 (+/-2g range)
```

```
        printf("X_g*100=%d\r\n",X_g);
        delay(100000);
    }
}
void setup()
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // enable GPIOA and GPIOB
    initClocks();
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while(dly--);
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0); // make sure GPIOA is turned on
    pinMode(GPIOA,2,2); // alternate function mode for PA2
    selectAlternateFunction(GPIOA,2,7); // AF7 = USART2 TX
    pinMode(GPIOA,15,2);
    selectAlternateFunction(GPIOA,15,3);
    RCC->APB1ENR1 |= (1 << 17); // turn on USART2

    const uint32_t CLOCK_SPEED=80000000;
    uint32_t BaudRateDivisor;

    BaudRateDivisor = CLOCK_SPEED/baudrate;
    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12); // disable over-run errors
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 =  (1 << 3);  // enable the transmitter and receiver
    USART2->CR1 |=  (1 << 2);  // enable the transmitter and receiver
    USART2->CR1 |= (1 << 0);
}
int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }
    while(len--)
    {
        eputc(*data);
        data++;
```

```
    }
    return 0;
}
void eputc(char c)
{
    while( (USART2->ISR & (1 << 6))==0); // wait for ongoing transmission to
finish
    USART2->TDR=c;
}
```

Result:
Acceleration values successfully read and printed. Values changed correctly with tilt.

11 March – Task 4: Basic Tilt Detection

Goal:
Use accelerometer to control LED output.

Code:
Threshold-based detection using X-axis.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>
#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS 16
#define WS_PIN   7   // PA7
#define I2C_TIMEOUT 20000

// -------------------- Accelerometer globals --------------------
uint16_t response;
int16_t x_accel;
int32_t X_g;

// -------------------- Function prototypes --------------------
void setup(void);
void initI2C(void);
void ResetI2C(void);
void I2CStart(uint8_t address, int rw, int nbytes);
void I2CReStart(uint8_t address, int rw, int nbytes);
void I2CWrite(uint8_t Data);
uint8_t I2CRead(void);
```

```c
void I2CStop(void);

void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_colour_all(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_x(void);

// -------------------- Main --------------------
int main(void)
{
    clock_init_80mhz();      // 80 MHz for WS2812 timings
    setup();                 // GPIO + clocks + I2C
    initSerial(9600);
    gpio_init_ws2812();
    dwt_init();

    ResetI2C();

    // Take accelerometer out of power down mode
    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);

    while (1)
    {
        X_g = read_bmi160_x();

        printf("X_g*100=%ld\r\n", X_g);

        // Thresholds based on your measured values:
        // stationary ~ 80
        // left ~ 1030
        // right ~ -920
```

```
        if (X_g > 600)
        {
            // Tilted left -> RED
            show_colour_all(0x00, 0x08, 0x00);
        }
        else if (X_g < -600)
        {
            // Tilted right -> BLUE
            show_colour_all(0x00, 0x00, 0x08);
        }
        else
        {
            // Near centre -> GREEN
            show_colour_all(0x08, 0x00, 0x00);
        }

        delay(100000);
    }
}

// -------------------- Read accelerometer --------------------
static int32_t read_bmi160_x(void)
{
    GPIOB->ODR |= (1 << 3);   // debug pin high

    I2CStart(0x69, WRITE, 1);
    I2CWrite(0x12);                 // X-axis low byte register
    I2CReStart(0x69, READ, 2);

    response = I2CRead();           // low byte
    x_accel = response;

    response = I2CRead();           // high byte
    x_accel = x_accel + (response << 8);

    I2CStop();

    GPIOB->ODR &= ~(1 << 3);        // debug pin low

    int32_t value = x_accel;        // preserve sign
    value = (value * 981) / 16384;  // scaled value

    return value;
}

// -------------------- General setup --------------------
void setup(void)
{
```

```c
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // GPIOA + GPIOB
    initClocks();
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

// -------------------- UART for printf --------------------
void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0); // GPIOA on

    pinMode(GPIOA, 2, 2);               // PA2 alternate function
    selectAlternateFunction(GPIOA, 2, 7); // USART2 TX

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17); // USART2 on

    const uint32_t CLOCK_SPEED = 80000000;
    uint32_t BaudRateDivisor = CLOCK_SPEED / baudrate;

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 = (1 << 3);   // TE
    USART2->CR1 |= (1 << 2);  // RE
    USART2->CR1 |= (1 << 0);  // UE
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
```

```c
        data++;
    }

    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

// -------------------- WS2812 clock --------------------
static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

// -------------------- WS2812 GPIO --------------------
static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
```

```c
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
```

```
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    // GRB order
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void show_colour_all(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        ws2812_send_pixel(g, r, b);
    }

    ws2812_reset();
}
```

Result:
System successfully detected left and right tilt. LED colour changed accordingly.

13 March – Task 5: Multi-Axis Control

Goal:
Include Y-axis for full directional control.

Code:
Added Y-axis reading and dominant-axis selection logic.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>
#include "eeng1030_lib.h"
#include "i2c.h"
```

```c
#define NUM_LEDS 16
#define WS_PIN   7

uint16_t response;
int16_t x_accel, y_accel;
int32_t X_g, Y_g;

// ---------- Prototypes ----------
void setup(void);
void initI2C(void);
void ResetI2C(void);
void I2CStart(uint8_t address, int rw, int nbytes);
void I2CReStart(uint8_t address, int rw, int nbytes);
void I2CWrite(uint8_t Data);
uint8_t I2CRead(void);
void I2CStop(void);

void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);
static void show_colour_all(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int32_t abs32(int32_t x);

// ---------- Main ----------
int main(void)
{
    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    dwt_init();

    ResetI2C();

    // Wake accelerometer
    I2CStart(0x69, WRITE, 2);
```

```
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);

    while (1)
    {
        X_g = read_bmi160_axis(0x12); // X low byte register
        Y_g = read_bmi160_axis(0x14); // Y low byte register

        printf("X_g*100=%ld  Y_g*100=%ld\r\n", X_g, Y_g);

        // choose dominant axis so diagonal tilts do not clash
        if (abs32(X_g) > abs32(Y_g))
        {
            if (X_g > 600)
            {
                // LEFT -> red
                show_colour_all(0x00, 0x08, 0x00);
            }
            else if (X_g < -600)
            {
                // RIGHT -> blue
                show_colour_all(0x00, 0x00, 0x08);
            }
            else
            {
                // near centre
                show_colour_all(0x08, 0x00, 0x00);
            }
        }
        else
        {
            if (Y_g > 600)
            {
                // FORWARD -> yellow = red + green
                show_colour_all(0x08, 0x08, 0x00);
            }
            else if (Y_g < -600)
            {
                // BACKWARD -> orange
                show_colour_all(0x02, 0x08, 0x00);
            }
            else
            {
                // near centre
                show_colour_all(0x08, 0x00, 0x00);
```

```
            }
        }

        delay(100000);
    }
}

// ---------- Read BMI160 axis ----------
static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    GPIOB->ODR |= (1 << 3);

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();         // low byte
    accel_raw = response;

    response = I2CRead();         // high byte
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    GPIOB->ODR &= ~(1 << 3);

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

static int32_t abs32(int32_t x)
{
    if (x < 0)
        return -x;
    return x;
}

// ---------- General setup ----------
void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initClocks();
    initI2C();
```

```c
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

// ---------- UART ----------
void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    const uint32_t CLOCK_SPEED = 80000000;
    uint32_t BaudRateDivisor = CLOCK_SPEED / baudrate;

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }
```

```c
    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

// ---------- Clock ----------
static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

// ---------- WS2812 ----------
static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
```

```c
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
```

```
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void show_colour_all(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        ws2812_send_pixel(g, r, b);
    }

    ws2812_reset();
}
```

Result:

System could now detect forward and backward tilt as well as left/right.

15 March – Task 6: Direction Mapping

Goal:

Map tilt direction to LED position on ring.

Code:

Used angle-based calculation (atan2) to determine LED index.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>
#include <math.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS   16
#define WS_PIN     7
```

```
#define TOP_LED    0      // change this if a different LED is physically at
the top
#define DEAD_ZONE  250    // if tilt magnitude is below this, show default top
3 LEDs

uint16_t response;
int16_t x_accel, y_accel;
int32_t X_g, Y_g;

// -------------------- Prototypes --------------------
void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y);
static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b);

// -------------------- Main --------------------
int main(void)
{
    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    dwt_init();

    ResetI2C();

    // Wake accelerometer
    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);
```

```c
    while (1)
    {
        X_g = read_bmi160_axis(0x12); // X low byte register
        Y_g = read_bmi160_axis(0x14); // Y low byte register

        printf("X_g*100=%ld  Y_g*100=%ld\r\n", X_g, Y_g);

        int centre_led = get_pointer_led(X_g, Y_g);

        // red pointer
        show_pointer_3(centre_led, 0x00, 0x08, 0x00);

        delay(80000);
    }
}

// -------------------- Accelerometer read --------------------
static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    GPIOB->ODR |= (1 << 3); // debug high

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();      // low byte
    accel_raw = response;

    response = I2CRead();      // high byte
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    GPIOB->ODR &= ~(1 << 3);   // debug low

    value = accel_raw;
    value = (value * 981) / 16384;  // scale to approx g*100

    return value;
}

// -------------------- Pointer logic --------------------
static int wrap_led(int n)
{
```

```
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
}

static int get_pointer_led(int32_t x, int32_t y)
{
    // If nearly flat, point to default top LED
    int32_t mag2 = x * x + y * y;
    if (mag2 < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    // Angle from x,y using atan2
    // atan2 gives radians from -pi to +pi
    double angle = atan2((double)y, (double)x);

    // Convert to LED index
    // We want:
    //   +Y -> TOP_LED
    //   +X -> 4 LEDs clockwise from top
    //   -Y -> 8 LEDs from top
    //   -X -> 12 LEDs from top
    //
    // Using angle_from_top = pi/2 - angle
    double angle_from_top = (M_PI / 2.0) - angle;

    // wrap to 0 .. 2pi
    if (angle_from_top < 0)
        angle_from_top += 2.0 * M_PI;
    if (angle_from_top >= 2.0 * M_PI)
        angle_from_top -= 2.0 * M_PI;

    int offset = (int)((angle_from_top / (2.0 * M_PI)) * NUM_LEDS + 0.5);
    offset %= NUM_LEDS;

    return wrap_led(TOP_LED + offset);
}

static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b)
{
    int led_a = wrap_led(centre_led - 1);
    int led_b = wrap_led(centre_led);
    int led_c = wrap_led(centre_led + 1);

    ws2812_reset();
```

```c
    for (int i = 0; i < NUM_LEDS; i++)
    {
        if (i == led_a || i == led_b || i == led_c)
            ws2812_send_pixel(g, r, b);
        else
            ws2812_send_pixel(0x00, 0x00, 0x00);
    }

    ws2812_reset();
}

// -------------------- General setup --------------------
void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // GPIOA + GPIOB
    initClocks();
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

// -------------------- UART --------------------
void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0); // GPIOA on

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7); // USART2 TX

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17); // USART2 on

    const uint32_t CLOCK_SPEED = 80000000;
    uint32_t BaudRateDivisor = CLOCK_SPEED / baudrate;

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 = (1 << 3);   // TE
    USART2->CR1 |= (1 << 2);  // RE
    USART2->CR1 |= (1 << 0);  // UE
}
```

```c
int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

// -------------------- 80 MHz clock --------------------
static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}
```

```c
    SystemCoreClock = 80000000;
}

// -------------------- WS2812 --------------------
static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));   // output mode

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2)); // very high speed
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));     // start low
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
```

```c
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    // WS2812 uses GRB order
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}
```

```c
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS   16
#define WS_PIN     7

#define TOP_LED    0
#define DEAD_ZONE  180

uint16_t response;
int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;
```

```
static const int16_t dir_x[NUM_LEDS] = {
     0,  383,  707,  924, 1000,  924,  707,  383,
     0, -383, -707, -924,-1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
   1000,  924,  707,  383,    0, -383, -707, -924,
  -1000, -924, -707, -383,    0,  383,  707,  924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b);

int main(void)
{
    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    dwt_init();

    ResetI2C();

    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);
```

```c
    while (1)
    {
        X_g = read_bmi160_axis(0x12);
        Y_g = read_bmi160_axis(0x14);
        Z_g = read_bmi160_axis(0x16);

        X_filt = (3 * X_filt + X_g) / 4;
        Y_filt = (3 * Y_filt + Y_g) / 4;
        Z_filt = (3 * Z_filt + Z_g) / 4;

        printf("X=%ld Y=%ld Z=%ld\r\n", X_filt, Y_filt, Z_filt);

        int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);

        show_pointer_3(centre_led, 0x00, 0x08, 0x00);

        delay(80000);
    }
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
```

```
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;

    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }

    return wrap_led(TOP_LED + best_led);
}

static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b)
{
    int led_a = wrap_led(centre_led - 1);
    int led_b = wrap_led(centre_led);
    int led_c = wrap_led(centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        if (i == led_a || i == led_b || i == led_c)
            ws2812_send_pixel(g, r, b);
        else
            ws2812_send_pixel(0x00, 0x00, 0x00);
    }

    ws2812_reset();
}

void setup(void)
```

```
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 80000000 / baudrate;
    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
```

```c
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void dwt_init(void)
```

```
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}
```

```
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}
```

Result:
LED pointer followed approximate direction of tilt. Some jitter observed.

17 March – Task 7: Pointer Display

Goal:
Improve visual output.

Code:
Implemented 3-LED pointer instead of single LED.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS   16
#define WS_PIN     7
#define BTN_PIN    9

#define TOP_LED    0
#define DEAD_ZONE  180

uint16_t response;
int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;

static const int16_t dir_x[NUM_LEDS] = {
     0,  383,  707,  924, 1000,  924,  707,  383,
     0, -383, -707, -924,-1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
   1000,  924,  707,  383,    0, -383, -707, -924,
```

```
  -1000, -924, -707, -383,    0,  383,  707,  924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void gpio_init_button(void);
static int button_pressed(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b);

int main(void)
{
    uint8_t colour_index = 0;
    uint8_t last_button_state = 0;

    uint8_t g_val = 0x00;
    uint8_t r_val = 0x08;
    uint8_t b_val = 0x00;

    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    gpio_init_button();
    dwt_init();

    ResetI2C();

    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();
```

```
    delay(1000000);

    while (1)
    {
        uint8_t current_button_state = button_pressed();

        if ((current_button_state == 1) && (last_button_state == 0))
        {
            delay(30000);

            if (button_pressed() == 1)
            {
                colour_index++;
                if (colour_index >= 4)
                    colour_index = 0;

                while (button_pressed() == 1) {}

                delay(30000);
            }
        }

        last_button_state = current_button_state;

        if (colour_index == 0)
        {
            g_val = 0x00;
            r_val = 0x08;
            b_val = 0x00;
        }
        else if (colour_index == 1)
        {
            g_val = 0x08;
            r_val = 0x08;
            b_val = 0x00;
        }
        else if (colour_index == 2)
        {
            g_val = 0x02;
            r_val = 0x08;
            b_val = 0x00;
        }
        else
        {
            g_val = 0x08;
            r_val = 0x00;
            b_val = 0x00;
        }
```

```
        X_g = read_bmi160_axis(0x12);
        Y_g = read_bmi160_axis(0x14);
        Z_g = read_bmi160_axis(0x16);

        X_filt = (3 * X_filt + X_g) / 4;
        Y_filt = (3 * Y_filt + Y_g) / 4;
        Z_filt = (3 * Z_filt + Z_g) / 4;

        printf("X=%ld Y=%ld Z=%ld colour=%d\r\n", X_filt, Y_filt, Z_filt,
colour_index);

        int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);

        show_pointer_3(centre_led, g_val, r_val, b_val);

        delay(80000);
    }
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
```

```
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;

    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }

    return wrap_led(TOP_LED + best_led);
}

static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b)
{
    int led_a = wrap_led(centre_led - 1);
    int led_b = wrap_led(centre_led);
    int led_c = wrap_led(centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        if (i == led_a || i == led_b || i == led_c)
            ws2812_send_pixel(g, r, b);
        else
            ws2812_send_pixel(0x00, 0x00, 0x00);
    }

    ws2812_reset();
}

void setup(void)
```

```
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 80000000 / baudrate;
    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
```

```c
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void gpio_init_button(void)
```

```c
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR |=  (1U << (BTN_PIN * 2));
}

static int button_pressed(void)
{
    if ((GPIOA->IDR & (1U << BTN_PIN)) == 0)
        return 1;
    else
        return 0;
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
```

```
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}
```

Result:
Output became clearer and easier to interpret.

19 March – Task 8: Filtering

Goal:
Reduce noise and jitter.

Code:
Applied low-pass filter to accelerometer readings.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS   16
```

```c
#define WS_PIN     7
#define BTN_PIN    9

#define TOP_LED    0
#define DEAD_ZONE  180

uint16_t response;
int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;

static const int16_t dir_x[NUM_LEDS] = {
     0,  383,  707,  924, 1000,  924,  707,  383,
     0, -383, -707, -924,-1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
   1000,  924,  707,  383,    0, -383, -707, -924,
  -1000, -924, -707, -383,    0,  383,  707,  924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void gpio_init_button(void);
static int button_pressed(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void initADC(void);
static int readADC(int chan);
static void initTimer2PWM(void);
static void setTimer2Duty(int duty);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b);

int main(void)
```

```
{
    uint8_t colour_index = 0;
    uint8_t last_button_state = 0;

    uint8_t g_val = 0x00;
    uint8_t r_val = 0x08;
    uint8_t b_val = 0x00;

    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    gpio_init_button();
    dwt_init();

    pinMode(GPIOA, 0, 3);   // PA0 analog input for pot
    pinMode(GPIOA, 3, 2);   // PA3 alternate function for PWM LED
    GPIOA->AFR[0] &= ~(0xF << (4 * 3));
    GPIOA->AFR[0] |=  (1U  << (4 * 3));   // AF1 = TIM2_CH4

    initADC();
    initTimer2PWM();

    ResetI2C();

    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);

    while (1)
    {
        uint8_t current_button_state = button_pressed();

        if ((current_button_state == 1) && (last_button_state == 0))
        {
            delay(30000);

            if (button_pressed() == 1)
            {
                colour_index++;
                if (colour_index >= 4)
                    colour_index = 0;

                while (button_pressed() == 1) {}
```

```
            delay(30000);
        }
    }

    last_button_state = current_button_state;

    int pot_value = readADC(5);
    int pwm_value = 200 + ((pot_value * 3600) / 4095);
    setTimer2Duty(pwm_value);

    if (colour_index == 0)
    {
        g_val = 0x00;
        r_val = 0x08;
        b_val = 0x00;
    }
    else if (colour_index == 1)
    {
        g_val = 0x08;
        r_val = 0x08;
        b_val = 0x00;
    }
    else if (colour_index == 2)
    {
        g_val = 0x02;
        r_val = 0x08;
        b_val = 0x00;
    }
    else
    {
        g_val = 0x08;
        r_val = 0x00;
        b_val = 0x00;
    }

    X_g = read_bmi160_axis(0x12);
    Y_g = read_bmi160_axis(0x14);
    Z_g = read_bmi160_axis(0x16);

    X_filt = (3 * X_filt + X_g) / 4;
    Y_filt = (3 * Y_filt + Y_g) / 4;
    Z_filt = (3 * Z_filt + Z_g) / 4;

    printf("POT=%d X=%ld Y=%ld Z=%ld colour=%d\r\n",
           pot_value, X_filt, Y_filt, Z_filt, colour_index);

    int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);
```

```
        show_pointer_3(centre_led, g_val, r_val, b_val);

        delay(80000);
    }
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;
```

```
    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }

    return wrap_led(TOP_LED + best_led);
}

static void show_pointer_3(int centre_led, uint8_t g, uint8_t r, uint8_t b)
{
    int led_a = wrap_led(centre_led - 1);
    int led_b = wrap_led(centre_led);
    int led_c = wrap_led(centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        if (i == led_a || i == led_b || i == led_c)
            ws2812_send_pixel(g, r, b);
        else
            ws2812_send_pixel(0x00, 0x00, 0x00);
    }

    ws2812_reset();
}

void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);
```

```
    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 80000000 / baudrate;
    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;
```

```c
    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void gpio_init_button(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR |=  (1U << (BTN_PIN * 2));
}

static int button_pressed(void)
{
    if ((GPIOA->IDR & (1U << BTN_PIN)) == 0)
        return 1;
    else
        return 0;
```

```c
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
```

```
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void initADC(void)
{
    RCC->AHB2ENR |= (1 << 13);
    RCC->CCIPR |= (1 << 29) | (1 << 28);
    ADC1_COMMON->CCR = ((0b01) << 16) + (1 << 22);

    ADC1->CR = (1 << 28);
    delay(100);

    ADC1->CR |= (1 << 31);
    while (ADC1->CR & (1 << 31)) {}

    ADC1->CFGR = (1 << 31);
}

static int readADC(int chan)
{
    ADC1->SQR1 = 0;
    ADC1->SQR1 |= (chan << 6);

    ADC1->ISR = (1 << 3);
    ADC1->CR |= (1 << 0);
    while ((ADC1->ISR & (1 << 0)) == 0) {}

    ADC1->CR |= (1 << 2);
    while ((ADC1->ISR & (1 << 3)) == 0) {}

    return ADC1->DR;
}

static void initTimer2PWM(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->CR1 = 0;
    TIM2->CCMR2 = (0b110 << 12) | (1 << 11);
    TIM2->CCER |= (1 << 12);
```

```
    TIM2->PSC = 79;
    TIM2->ARR = 1000 - 1;
    TIM2->CCR4 = 0;

    TIM2->EGR |= (1 << 0);
    TIM2->CR1 |= (1 << 7);
    TIM2->CR1 |= (1 << 0);
}

static void setTimer2Duty(int duty)
{
    int arrvalue = (duty * (TIM2->ARR + 1)) / 4095;

    if (arrvalue < 5)
        arrvalue = 5;

    if (arrvalue > (TIM2->ARR - 5))
        arrvalue = TIM2->ARR - 5;

    TIM2->CCR4 = arrvalue;
}
```

21 March – Final System Integration

Goal:
Complete system with all features.

Code:
Integrated tilt detection, LED mapping, filtering, and user inputs.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS   16
#define WS_PIN     7
#define BTN_PIN    9

#define TOP_LED    0
#define DEAD_ZONE  180
```

```
#define SAVE_THRESHOLD 200

uint16_t response;
int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;

int saved_mode = 0;
int saved_centre_led = 0;
int save_latched = 0;

static const int16_t dir_x[NUM_LEDS] = {
     0,  383,  707,  924, 1000,  924,  707,  383,
     0, -383, -707, -924,-1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
   1000,  924,  707,  383,    0, -383, -707, -924,
  -1000, -924, -707, -383,    0,  383,  707,  924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void gpio_init_button(void);
static int button_pressed(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void initADC(void);
static int readADC(int chan);
static void initTimer2PWM(void);
static void setTimer2Duty(int duty);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active, uint8_t g, uint8_t r, uint8_t b);
```

```c
int main(void)
{
    uint8_t colour_index = 0;
    uint8_t last_button_state = 0;

    uint8_t g_val = 0x00;
    uint8_t r_val = 0x08;
    uint8_t b_val = 0x00;

    clock_init_80mhz();
    setup();
    initSerial(9600);
    gpio_init_ws2812();
    gpio_init_button();
    dwt_init();

    pinMode(GPIOA, 0, 3);
    pinMode(GPIOA, 3, 2);
    GPIOA->AFR[0] &= ~(0xF << (4 * 3));
    GPIOA->AFR[0] |=  (1U << (4 * 3));

    initADC();
    initTimer2PWM();

    ResetI2C();

    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
    I2CStop();

    delay(1000000);

    while (1)
    {
        uint8_t current_button_state = button_pressed();

        if ((current_button_state == 1) && (last_button_state == 0))
        {
            delay(30000);

            if (button_pressed() == 1)
            {
                colour_index++;
                if (colour_index >= 4)
                    colour_index = 0;

                while (button_pressed() == 1) {}
```

```
            delay(30000);
        }
    }

    last_button_state = current_button_state;

    X_g = read_bmi160_axis(0x12);
    Y_g = read_bmi160_axis(0x14);
    Z_g = read_bmi160_axis(0x16);

    X_filt = (3 * X_filt + X_g) / 4;
    Y_filt = (3 * Y_filt + Y_g) / 4;
    Z_filt = (3 * Z_filt + Z_g) / 4;

    int pot_value = readADC(5);
    int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);

    if (pot_value <= SAVE_THRESHOLD)
    {
        setTimer2Duty(0);

        if (save_latched == 0)
        {
            saved_centre_led = centre_led;
            saved_mode = 1;
            save_latched = 1;
        }
    }
    else
    {
        setTimer2Duty(pot_value);
        saved_mode = 0;
        save_latched = 0;
    }

    if (colour_index == 0)
    {
        g_val = 0x00;
        r_val = 0x08;
        b_val = 0x00;
    }
    else if (colour_index == 1)
    {
        g_val = 0x08;
        r_val = 0x08;
        b_val = 0x00;
    }
```

```
        else if (colour_index == 2)
        {
            g_val = 0x02;
            r_val = 0x08;
            b_val = 0x00;
        }
        else
        {
            g_val = 0x08;
            r_val = 0x00;
            b_val = 0x00;
        }

        printf("POT=%d X=%ld Y=%ld Z=%ld colour=%d saved=%d saved_led=%d
live_led=%d\r\n",
               pot_value, X_filt, Y_filt, Z_filt, colour_index, saved_mode,
saved_centre_led, centre_led);

        show_live_and_saved(centre_led, saved_centre_led, saved_mode, g_val,
r_val, b_val);

        delay(80000);
    }
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}
```

```c
static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;

    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }

    return wrap_led(TOP_LED + best_led);
}

static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active, uint8_t g, uint8_t r, uint8_t b)
{
    int live_a = wrap_led(live_centre_led - 1);
    int live_b = wrap_led(live_centre_led);
    int live_c = wrap_led(live_centre_led + 1);

    int save_a = wrap_led(saved_centre_led - 1);
    int save_b = wrap_led(saved_centre_led);
    int save_c = wrap_led(saved_centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
```

```
        uint8_t gg = 0x00;
        uint8_t rr = 0x00;
        uint8_t bb = 0x00;

        if (saved_active && (i == save_a || i == save_b || i == save_c))
        {
            gg = 0x00;
            rr = 0x00;
            bb = 0x08;
        }

        if (i == live_a || i == live_b || i == live_c)
        {
            gg = g;
            rr = r;
            bb = b;
        }

        ws2812_send_pixel(gg, rr, bb);
    }

    ws2812_reset();
}

void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
```

```c
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 80000000 / baudrate;
    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
```

```c
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |=  (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |=  (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void gpio_init_button(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR |=  (1U << (BTN_PIN * 2));
}

static int button_pressed(void)
{
    if ((GPIOA->IDR & (1U << BTN_PIN)) == 0)
        return 1;
    else
        return 0;
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}
```

```c
static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}
```

```c
static void initADC(void)
{
    RCC->AHB2ENR |= (1 << 13);
    RCC->CCIPR |= (1 << 29) | (1 << 28);
    ADC1_COMMON->CCR = ((0b01) << 16) + (1 << 22);

    ADC1->CR = (1 << 28);
    delay(100);

    ADC1->CR |= (1 << 31);
    while (ADC1->CR & (1 << 31)) {}

    ADC1->CFGR = (1 << 31);
}

static int readADC(int chan)
{
    ADC1->SQR1 = 0;
    ADC1->SQR1 |= (chan << 6);

    ADC1->ISR = (1 << 3);
    ADC1->CR |= (1 << 0);
    while ((ADC1->ISR & (1 << 0)) == 0) {}

    ADC1->CR |= (1 << 2);
    while ((ADC1->ISR & (1 << 3)) == 0) {}

    return ADC1->DR;
}

static void initTimer2PWM(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->CR1 = 0;
    TIM2->CCMR2 = (0b110 << 12) | (1 << 11);
    TIM2->CCER |= (1 << 12);

    TIM2->PSC = 79;
    TIM2->ARR = 1000 - 1;
    TIM2->CCR4 = 0;

    TIM2->EGR |= (1 << 0);
    TIM2->CR1 |= (1 << 7);
    TIM2->CR1 |= (1 << 0);
}
```

```c
static void setTimer2Duty(int duty)
{
    int arrvalue;

    if (duty < 0)
        duty = 0;

    if (duty > 4095)
        duty = 4095;

    if (duty <= SAVE_THRESHOLD)
    {
        TIM2->CCR4 = 0;
        return;
    }

    arrvalue = (duty * (TIM2->ARR + 1)) / 4095;
    TIM2->CCR4 = arrvalue;
}
```

Result:
Final system fully functional:

- Accurate tilt detection
- Stable LED output
- Smooth pointer movement
- Reliable performance